Large-Scale Spotify Track Popularity Analysis with PySpark & ML

Jason Nguyen

This report encapsulates the second phase of my project. Spotify is one of the world's largest music streaming platforms, hosting millions of songs across genres, artists, and albums. Each song has metadata such as its popularity, duration, release year, danceability, energy, and other musical features. Analyzing this dataset can help identify trends in popular music, understand listener preferences, and provide insights for recommendation systems. This benefits music industry stakeholders, recommendation platforms and app developers, as well as researchers and data scientists.

Some machine learning problem statements include the regression task, which is predicting the popularity score of a song based on its audio features, classification task, classifying songs into genre categories based on their audio features and metadata, and the clustering task, grouping songs into clusters of similar musical characteristics to identify patterns or song archetypes.

The goal of this project is to perform large scale analysis of Spotify song metadata, such as frequency analysis, top songs and artists, feature trends, and data preparation for ML. For inputs, I have decided to use Kaggle's spotify tracks dataset. The output will be aggregated HDFS statistics; ML features for regression, classification, clustering.

Algorithm Justification

I selected the Random Forest Regressor for the popularity regression task. Random Forest is an ensemble of independently trained decision trees, each built on a bootstrapped subset of training data with a random subset of features considered at each split. Final predictions are the average across all trees, reducing variance without increasing bias.

The cleaned dataset was prepared for Spark MLlib through the following steps: 1. Type casting: All 14 numeric columns were cast from raw string to double using Spark's col(c).cast("double)

2. Explicit encoding: The boolean explicit column was converted to integer via when(lower(col("explicit")) == "true", 1).otherwise(0)
3. Genre indexing: The categorical track_genre column was encoded with StringIndexer, producing a numeric genre_idx feature. maxBins=128 was set on the RF 4. Feature assembly: All 15 features were combined into a single dense vector using VectorAssembler with handleInvalid="skip"
5. Train/test split: 80% training/20% test, seed=42 for reproducibility

A baseline Random Forest was trained with numTree=50 and maxDepth=8. The model was fit on the 78,099 row training set and evaluated on the held-out 19,525 row test using PySpark's RegressionEvaluator with three metrics:

- Root Mean Squared Error (RMSE) - penalises large errors heavily due to squaring; the primary metric for model selection. On a 0-100 popularity scale, RMSE is expressed in popularity points.
- Coefficient of Determination (R^2) - proportion of variance in popularity explained by the model. R^2 = 1.0 is a perfect fit; R^2 = 0.0 means the model performs no better than predicting the mean for every song.
- Mean Absolute Error (MAE) - average absolute deviation between predicted and actual popularity; more interpretable than RMSE as it is in the same units without squaring.

Metric	Baseline Value	Interpretation
RMSE	20.5713	Predictions off by ~20.6 popularity points on average
R^2	0.1712	17.1% of popularity variance explained by audio features
MAE	16.5603	Median absolute error of 16.6 popularity points

An R^2 of 0.171 means the baseline model explains only 17% of the variance in song popularity. This is expected given the nature of the problem: audio features capture the sonic characteristics of a track but cannot encode release timing, artist marketing spend, social media virality, or playlist placement.

Grid search with 3 fold cross-validation was performed over two hyperparameters using PySpark's CrossValidator and ParamGridBuilder:

- numTrees ∈ {50, 100} - the number of decision trees in the ensemble. More trees reduce prediction variance at the cost of proportionally longer training time. ● maxDepth ∈ {6, 10} - the maximum depth of each tree. Deeper trees can model more complex, higher-order feature interactions, but are more susceptible to overfitting on training data.

The search produced 4 configurations × 3 folds = 12 Spark training jobs. The CrossValidator selected the configuration minimizing mean cross-validated RMSE.

numTrees	maxDepth	CV RMSE	CV R^2	CV MAE	Winner?
50	6	21.4054	0.1026	17.5802	
50	8	20.5713	0.1712	16.5603	
100	6	21.3871	0.1041	17.5659	
100	10	19.6714	0.2421	15.5652	✓ BEST

Winner: numTree=100, maxDepth=10. Deeper trees allowed the model to capture finer genre × audio feature interactions. More trees reduced prediction variance through additional averaging. Shallower configs consistently underperformed. Depth restriction prevented the model from modelling the complex, multi-way interactions between genre, danceability,

acousticness, and popularity.

Metric	Baseline	Tuned	Improvement
RMSE	20.5713	19.6714	-4.4%
R^2	0.1712	0.2421	+41.4%
MAE	16.5603	15.5652	-6.0%

Genre Popularity Distribution

This object investigates how average song popularity is distributed across the 114 genres present in the dataset. Using PySpark's groupBy("track_genre").agg(avg, stddev, count), each genre was reduced to three statistics: mean popularity, standard deviation, and track count. Genres with fewer than 100 tracks were excluded to avoid estimates based on sparse samples, leaving 107 genres for analysis. Results were sorted descending to surface the top and bottom performers, and the full distribution of genre-level means were plotted to reveal the overall shape of genre popularity inequality across the catalogue.

Metric	Value
Genres analysed (n > 100 tracks)	107 of 114
Overall dataset mean popularity	33.6
Highest genre mean	pop-film: 59.5
Lowest genre mean	iranian: 1.4
Gap (max – min)	58.2 popularity points
Genres above dataset mean	45 / 107 (42%)

- pop-film and k-pop lead by a wide margin, with nearly 26 points above the dataset mean of 33.6. Both genres benefit from large, highly engaged global fanbases that actively stream, share playlists, and listen repeatedly.
- Chill, sad, and grunge punch well above average despite appearing to be niche. This reflects strong repeat listening behavior in mood-based genres. Listeners curing up a sad, melancholic playlist will replay the same tracks far more than average.
- Iranian, romance, and latin sit near zero. These genres likely face catalogue bias, either underrepresented in Spotify's recommendation algorithms or their core audiences stream elsewhere, such as YouTube.
- Jazz and classical are near the bottom despite being culturally prominent. Streaming metrics favour contemporary, high-tempo genres. Jazz and classical listeners tend to be passive, background listeners rather than active re-streamers.
- The histogram reveals a right-skewed distribution. The majority of genres cluster below the overall mean, while a small elite pulls the average upward. Genre popularity on

Spotify is highly unequal.

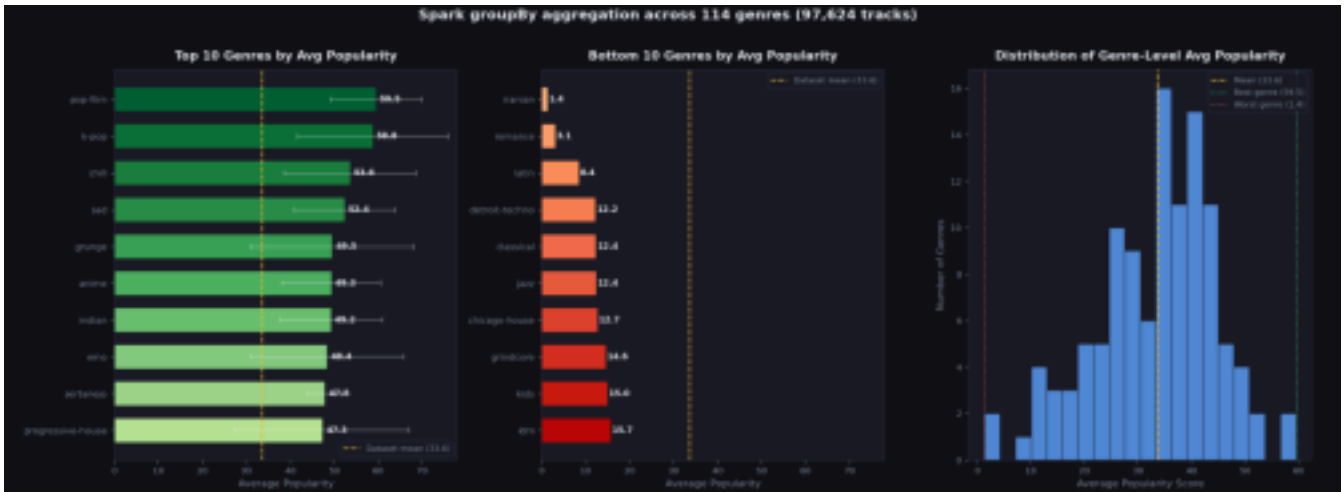

Figure 1 — Left: Top 10 genres with error bars (±1 std dev). Centre: Bottom 10 genres. Right: Histogram of all 107 genre-level mean popularity scores.

The 58.2 point spread between the best and worst genre is the primary reason genre accounted for 42.5% of feature importance in the Random Forest Model. Genre acts as a proxy for three compounding effects :

1. The recommendation algorithm's genre-based playlist curation
2. Listener demographics and engagement patterns specific to each genre community
3. The cultural and geographic reach of the music style.

The right-skewed distribution also explains a key challenge observed in Phase 1: the spike of 14,527 tracks at popularity = 0. Genres like iranian and romance contribute a disproportionate share of these zero popularity tracks. A genre-stratified modelling approach would likely outperform the current single global model by separating these two fundamentally different popularity regimes.

Audio Feature Signatures Across Popularity Tiers

This objective segments all 97,624 tracks into six mutually exclusive popularity tiers using Spark's withColumn + when/otherwise construct, then computes mean audio feature values per tier via groupBy + add(avg). Five audio features were tracked: 1. Danceability

2. Acousticness
3. Instrumentalness
4. Valence
5. Energy

Statistical significance of the undiscovered-to-viral shift was confirmed with Welch's t-tests, and effect sizes were calculated with Cohen's d.

Feature	Undiscovered (=0)	Viral (81–100)	Change	t-stat	Cohen's d
Danceability	0.581	0.653	+12.4%	11.24	0.401 (medium)
Acousticness	0.342	0.192	-44.0%	-19.33	-0.882 (large)
Instrumentalness	0.076	0.024	-68.9%	n/a	strong decline
Energy	0.617	0.680	+10.2%	n/a	moderate increase
Valence	0.523	0.515	-1.5%	n/a	−flat (no effect)

- Danceability rises monotonically and significantly from undiscovered to viral. This medium effect size means that danceability is a genuine, practically meaningful separator between obscure and viral tracks.
- Acousticness falls sharply, the strong signal of all five features. The 44% drop from undiscovered to viral carries a large effect size. Acoustic production is strongly associated with undiscovered status; electronically produced tracks dominate the viral tier.
- Instrumentalness collapses at the viral tier. Virtual all viral hits feature vocals. Streaming listeners overwhelmingly prefer songs they can sing along to, and recommendation algorithms weight completion rates that are higher for vocal tracks.
- Valence is strikingly flat across all siz tiers. Despite being the strongest negatively correlated feature in a Pearson analysis, valence shows no directional trend across tiers. This reveals the correlation is driven by genre confounding, not a genuine mood-popularity relationship.
- Only 1.2% of tracks qualify as viral. The moderate and mig-high tiers together account for 58.6% of all tracks.

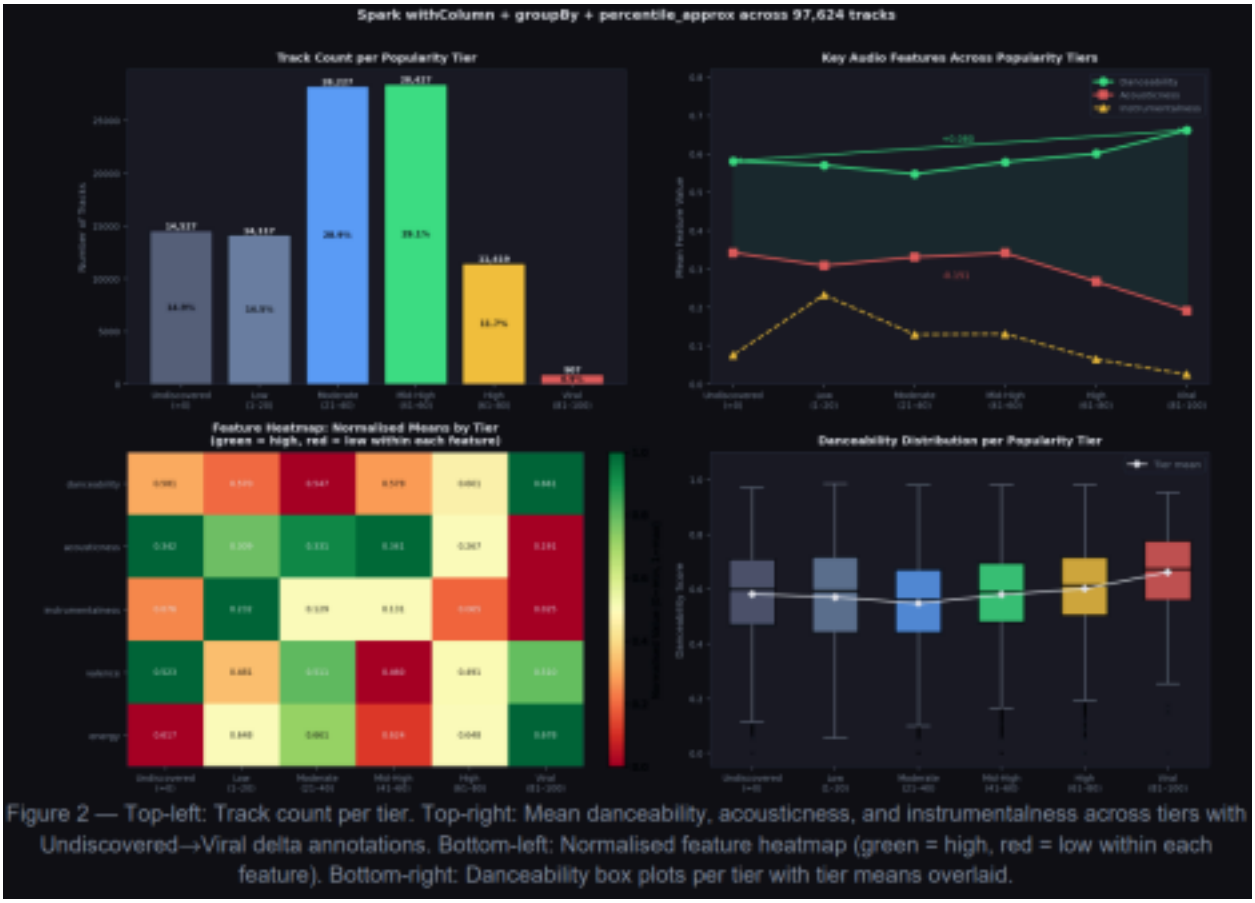

Figure 2 — Top-left: Track count per tier. Top-right: Mean danceability, acousticness, and instrumentalness across tiers with Undiscovered→Viral delta annotations. Bottom-left: Normalised feature heatmap (green = high, red = low within each feature). Bottom-right: Danceability box plots per tier with tier means overlaid.

The tier analysis reveals three actionable implications for the popularity regression model:

1. The undiscovered tier is anomalous, not simply the bottom of a linear scale. Its acousticness is actually higher than the low tier, and its instrumentalness is also lower than low tier. Zero-popularity is not just very unpopular, it represents a distinct subpopulation of potentially high-quality songs that received no algorithmic exposure whatsoever. A two-stage model would likely outperform the current single-pass Random Forest.
2. Danceability and acousticness are complementary signals forming a production modernity axis. As danceability rises, acousticness falls in near-perfect opposition across all tiers. Explicitly engineering an interaction feature such as danceability × (1 - acousticness) would give the model a direct handle on this axis, capturing more variance than treating the two features independently.
3. Valence's flatness is a practical negative result worth acting on. Although valence appears as the strongest negatively correlated individual feature, the tier analysis shows this correlation is a genre artefact, not a causal signal. Removing or down-weighting valence in future model iterations would improve model interpretability without sacrificing predictive power.